

Nuvem Pessoal Simplificada: API REST em Node.js integrada ao MinIO para armazenamento de objetos

Carlos Henrique de Souza Santana; Leanderson Torres de Souza; Gerson Brenner da Silva Farias

Curso de Análise e Desenvolvimento de Sistemas - ESBAM | Manaus - AM | 10 de setembro de 2026

Resumo. Este trabalho apresenta o desenvolvimento de uma solução simplificada de armazenamento de objetos, composta por uma API REST em Node.js integrada ao MinIO e executada em containers Docker. A proposta foi estruturada para reproduzir, em ambiente local, conceitos associados ao modelo de Object Storage compatível com a interface S3, com persistência por volume Docker e criação automatizada do bucket da aplicação. O backend foi organizado em camadas de configuração, rotas, controladores e serviços, permitindo realizar envio, listagem e recuperação de arquivos por meio de requisições HTTP. A validação foi realizada com chamadas HTTP em ambiente local, incluindo testes de upload, listagem e download, além da execução integrada dos serviços por Docker Compose. Os resultados demonstraram funcionamento consistente da comunicação entre API e MinIO e persistência dos objetos no bucket configurado.

Palavras-chave: Object Storage; MinIO; Node.js; Docker; Docker Compose; API REST; S3.

Abstract. This paper presents the development of a simplified object storage solution composed of a Node.js REST API integrated with MinIO and executed through Docker containers. The solution reproduces, in a local environment, concepts associated with S3-compatible Object Storage, using Docker volumes for persistence and automated creation of the application bucket. The backend was organized into configuration, routes, controllers and services, enabling file upload, listing and retrieval through HTTP requests. Validation was performed with HTTP requests in a local environment, including upload, listing and download tests, as well as integrated execution of the services through Docker Compose. The results showed consistent communication between the API and MinIO and successful persistence of the stored objects.

1. Introdução e Fundamentação Teórica

O armazenamento de dados pode ser organizado em diferentes modelos, entre os quais se destacam File Storage, Block Storage e Object Storage. No File Storage, os dados são estruturados em arquivos e diretórios, favorecendo cenários em que usuários e aplicações trabalham com uma hierarquia de pastas. O Block Storage disponibiliza blocos de armazenamento para sistemas que precisam administrar estruturas de dados em nível mais baixo, sendo comum em discos virtuais e bancos de dados. Já o Object Storage organiza os conteúdos como objetos identificados por chaves, acompanhados de metadados e armazenados em buckets. Essa abordagem é adequada para grandes volumes de arquivos, conteúdo multimídia, backups e documentos, pois reduz a dependência de uma hierarquia tradicional de diretórios.

Neste projeto, o MinIO foi adotado por disponibilizar uma implementação de Object Storage compatível com a interface S3, permitindo trabalhar localmente com conceitos utilizados em plataformas de nuvem. O objetivo não foi reproduzir toda a complexidade de uma infraestrutura de produção, mas criar um ambiente controlado para compreender a relação entre uma API REST e um serviço de armazenamento de objetos. O uso de Docker e Docker Compose também favorece reprodutibilidade, isolamento dos componentes e facilidade de inicialização do ambiente.

2. Objetivos e Escopo da Solução

O objetivo técnico da atividade foi desenvolver uma API RESTful em Node.js capaz de receber arquivos binários, armazená-los em um bucket MinIO e disponibilizar operações para listagem e recuperação. Paralelamente, o ambiente deveria ser orquestrado por Docker Compose, com credenciais recebidas por variáveis de ambiente, persistência por volume e criação automática do bucket.

O projeto foi delimitado a um backend sem interface gráfica. A comunicação é realizada por HTTP, utilizando multipart/form-data no envio de arquivos. A aplicação foi estruturada com Express, Multer para tratamento de upload em memória, dotenv para leitura das variáveis de ambiente e o SDK do MinIO para comunicação com o armazenamento de objetos.

3. Arquitetura da Solução

A arquitetura foi dividida em quatro componentes lógicos: cliente HTTP, API Node.js, ambiente Docker Compose e servidor MinIO. O cliente HTTP envia requisições para a API. A API valida a requisição e encaminha a operação ao serviço de arquivos. O serviço, por sua vez, utiliza o cliente MinIO para gravar, listar ou recuperar objetos no bucket da aplicação. O Docker Compose mantém os serviços em uma rede compartilhada e utiliza um volume nomeado para garantir persistência dos dados.

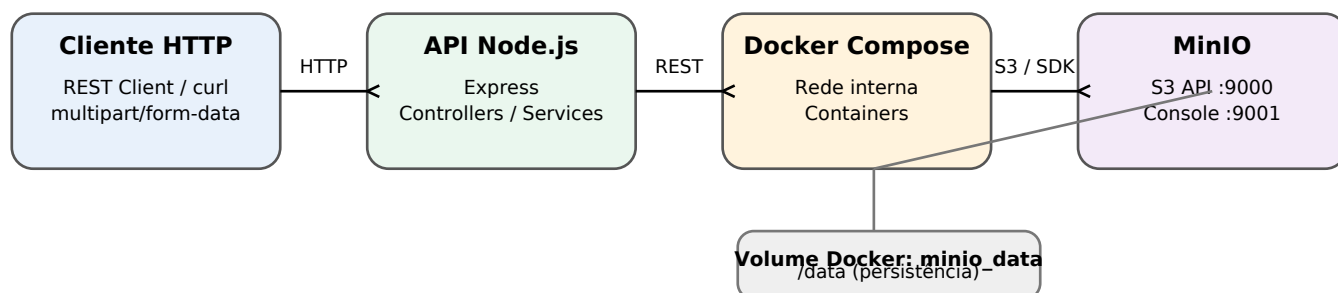


Figura 1 - Arquitetura conceitual da solução implementada.

No ambiente final, a API é executada no container nuvem-api e exposta no host pela porta 3001, enquanto o processo da aplicação escuta internamente na porta 3000. O MinIO disponibiliza a API S3 na porta 9000 e o Console Web na porta 9001. A comunicação entre os containers ocorre pelo nome do serviço Docker, de modo que a API utiliza o host minio para acessar a porta 9000 internamente.

O arquivo docker-compose.yml também inclui um serviço auxiliar minio-mc. Esse serviço aguarda o MinIO estar saudável e utiliza o MinIO Client para criar o bucket arquivos-pessoais, utilizando a opção --ignore-existing para permitir reinicializações sem erro quando o bucket já estiver criado. A persistência dos objetos é garantida pelo volume minio_data, montado em /data.

4. Metodologia e Implementação

4.1. Estrutura do backend

O backend foi organizado de forma modular. O arquivo app.js configura o Express, CORS e as rotas da aplicação. O arquivo server.js inicializa o servidor e verifica a existência do bucket antes de liberar o serviço. A configuração do cliente MinIO foi separada em src/config/minio.js. As rotas encontram-se em src/routes/filesRoutes.js, enquanto src/controllers/filesController.js concentra o tratamento das requisições. A comunicação efetiva com o armazenamento foi encapsulada em src/services/filesService.js.

4.2. Operações implementadas

Operação	Endpoint	Descrição
Upload	POST /files/upload	Recebe multipart/form-data e grava o objeto no bucket.
Listagem	GET /files	Retorna os objetos encontrados no bucket, com metadados disponíveis pela API.
Download	GET /files/:filename	Recupera o objeto e realiza streaming da resposta ao cliente.

No envio, o Multer utiliza memoryStorage(), mantendo temporariamente o conteúdo em memória antes de a aplicação executar putObject no MinIO. Na recuperação, a API utiliza getObject e direciona o stream recebido diretamente para a resposta HTTP, evitando a necessidade de montar previamente um arquivo local para download.

5. Orquestração com Docker e MinIO

A infraestrutura foi definida em Docker Compose. O serviço minio utiliza a imagem oficial do MinIO, expõe as portas 9000 e 9001, monta o volume minio_data em /data e recebe MINIO_ROOT_USER e MINIO_ROOT_PASSWORD por variáveis de ambiente. O serviço minio-mc depende da condição de saúde do MinIO e executa o comando de criação do bucket. A API possui uma imagem própria construída a partir de um Dockerfile baseado em node:22-alpine.

Um ajuste importante durante a dockerização foi a diferença entre localhost e o nome do serviço dentro da rede Docker. Quando a API era executada diretamente no sistema operacional, localhost:9000 era suficiente para localizar o MinIO. Dentro do container, entretanto, o endpoint passou a ser minio:9000. Essa alteração foi configurada por MINIO_ENDPOINT=minio no serviço api.

6. Resultados e Testes

A validação funcional foi feita por meio de requisições HTTP realizadas no terminal. Primeiro, a API respondeu à rota raiz com uma mensagem confirmando sua execução. Em seguida, a listagem de arquivos demonstrou que o serviço Node.js conseguia consultar o bucket do MinIO.

Teste	Resultado observado
GET /	HTTP retornou a mensagem de que a API estava funcionando.
GET /files	Listagem retornou objetos armazenados no bucket.
POST /files/upload	Upload retornou mensagem de sucesso e nome do arquivo enviado.
GET /files	Após upload, o novo objeto apareceu na listagem.
Download	Arquivo de teste pôde ser recuperado pelo endpoint de arquivo.

Durante os testes, foram utilizados arquivos pequenos de validação, incluindo exemplos com 13 e 15 bytes. Não foi realizado um teste sistemático de limite máximo de upload nem uma medição formal de latência; por isso, esses indicadores não são apresentados como métricas quantitativas do sistema. O foco dos testes executados foi confirmar a funcionalidade ponta a ponta entre cliente, API e MinIO.

Também foi validada a execução integrada por Docker Compose. Após a construção da imagem da API, o comando docker compose ps mostrou o MinIO em estado healthy e o container da API em execução. A comunicação com o endpoint localhost:3001 foi então validada com sucesso.

7. Desafios Técnicos e Lições Aprendidas

O primeiro desafio ocorreu na listagem de objetos. A chamada inicial ao SDK utilizou um parâmetro booleano em posição incorreta, provocando um erro de prefixo inválido. A solução consistiu em utilizar a assinatura correta de listObjects(bucketName, "", true), permitindo listar recursivamente os objetos a partir de um prefixo vazio.

Outro obstáculo surgiu na dockerização da API. A primeira tentativa de subir o serviço utilizou a porta 3000 no host, mas ela já estava ocupada pelo processo Node executado fora do Docker. Em vez de interromper a configuração funcional existente, foi feito o mapeamento 3001:3000, preservando a porta interna da aplicação e liberando a porta 3001 no computador.

Também foi necessário tratar a descoberta do MinIO dentro da rede Docker. A aplicação não poderia continuar usando localhost como hostname interno, pois isso apontaria para o próprio container da API. A substituição por minio demonstrou, na prática, a importância da resolução de nomes dos serviços em uma arquitetura multi-container.

Entre as principais lições aprendidas destacam-se a importância da modularização do backend, da separação entre configuração e lógica de negócio, do uso de variáveis de ambiente para credenciais e da verificação incremental do ambiente. A validação por pequenas etapas reduziu o tempo de diagnóstico e tornou possível localizar erros de forma objetiva.

8. Conclusão

O projeto Nuvem Pessoal Simplificada demonstrou a integração prática entre uma API REST em Node.js, um serviço de Object Storage compatível com S3 e uma infraestrutura orquestrada com Docker Compose. A solução foi capaz de receber arquivos por HTTP, persistir os objetos no MinIO, listar os conteúdos do bucket e recuperar arquivos por streaming.

Além do funcionamento da aplicação, o trabalho permitiu aplicar conceitos de containers, redes internas, volumes persistentes, variáveis de ambiente, modularização de código e testes de integração. Os resultados obtidos atendem ao objetivo de construir uma infraestrutura local reproduzível para armazenamento de objetos. Como evolução futura, a solução pode incorporar geração de URLs pré-assinadas, validação de tipos de arquivo, limites configuráveis de upload, autenticação de usuários e métricas automatizadas de desempenho.

9. Referências Bibliográficas

AMAZON WEB SERVICES. Amazon S3 - Object Storage. Documentação oficial. Disponível em: <https://aws.amazon.com/s3/>. Acesso em: 10 set. 2026.

MINIO. MinIO Object Storage. Documentação oficial. Disponível em: <https://min.io/docs/>. Acesso em: 10 set. 2026.

DOCKER. Docker Documentation. Documentação oficial. Disponível em: <https://docs.docker.com/>. Acesso em: 10 set. 2026.

NODE.JS. Node.js Documentation. Documentação oficial. Disponível em: <https://nodejs.org/docs/>. Acesso em: 10 set. 2026.

EXPRESS. Express - Fast, unopinionated, minimalist web framework for Node.js. Documentação oficial. Disponível em: <https://expressjs.com/>. Acesso em: 10 set. 2026.

Observação metodológica: os resultados e limitações descritos neste artigo correspondem aos testes efetivamente realizados durante a implementação do projeto. Métricas de desempenho detalhadas e testes de limite máximo de upload não foram medidos de forma sistemática neste ciclo.